PorkSort: Pass/Fail Inspection of Pork Rasher Packaging

Oct 25, 2026 · Hashim Navas

Hashim Navas, student ID [add] · MMA3001 Numerical Methods and Machine Learning, Semester 2 2026, Monash University · Individual project, Dataset 3 · Repository: github.com/hashim-edu/mma3001-project-pork_sort

Summary

PorkSort inspects camera frames of sealed pork rasher trays on a packing line. For each frame it returns PASS or FAIL, the defects found and where they are, so that suspect packs can be routed to manual re-inspection. A YOLOv8n detector, fine-tuned on 869 distinct conveyor frames and run with ONNX Runtime on a laptop CPU, gives each frame a defect score. The threshold is chosen to minimise expected cost, with a missed defect costed at ten re-checks. The provided test split shares trays with the training data, so I built a blocked split with a 30-frame buffer. On its 174 test frames the detector ranks frames with a ROC AUC of 0.887 (95% CI 0.810 to 0.947), 0.148 above the best classical model (paired 95% CI 0.068 to 0.235). It does not meet my acceptance criteria. At the design cost ratio no model beats re-checking every pack. The detector only pays off when a miss costs about as much as a re-check: it then passes 56% of good frames automatically and lets 3.6% of defective frames through. It runs in 41 ms per frame on an Apple M4 CPU, well inside the 300 to 500 ms per pack of a 120 to 200 pack per minute line.

1 Engineering problem

Pork rashers are packed in plastic trays and sealed with lidding film on automated tray-sealing lines. Meat lines typically run at about 120 packs per minute [1], and recent machines are rated up to 200 [2]. A pack that leaves the line unsealed, or with damaged film, risks spoilage and a product recall. Loose or twisted meat is a presentation fault that leads to customer complaints. At two to three packs per second, consistent visual inspection by people is hard to sustain over a shift. Automating it is a machine-vision and mechatronics problem, part of the instrumentation and control of a manufacturing line.

The Dataset 3 brief poses an open problem and asks for a narrower, achievable one. I chose:

> *Given one RGB frame from the overhead camera, decide whether the visible tray should PASS or FAIL packaging inspection, and show where any defect is.*

The intended use is decision support. Every FAIL frame is shown to an operator with its defect boxes, and the operator confirms the decision. The system never rejects product on its own. Its value is concentrating human attention on the frames most likely to be defective.

The scope is limited by what the data can support:

- **Per frame, not per tray.** Frames carry no tray identifier. A tray stays in view for about 12 frames (Section 2), so a tray-level decision would need tracking, which I left for future work.
- **Packaging and placement only.** The labels cover five defects: unsealed film, other packaging errors, loose meat, twisted meat, and wrinkles. There are no labels for meat quality, such as fat content, so that part of the open problem is out of scope.
- **One line, one camera, one lighting setup.** All frames come from the same conveyor recording, so no claim is made about other lines.

FAIL means any of the five defects is present. I treated all five as equally serious, because the safe default for an unconfirmed defect is a human check.

2 Inputs, outputs and data

2.1 Top-level inputs and outputs

Item	Format and units	Domain	Meaning and use
Input frame	JPEG, PNG or BMP; RGB, 8 bits per channel	720 x 540 px overhead conveyor frame. Other sizes are letterboxed and flagged	One camera frame of the line
Defect score s	Dimensionless	0 to 1	Highest confidence among detected defect boxes; ranks how suspicious the frame is
Threshold τ	Dimensionless	0 to 1, fixed from validation data	The frame FAILs when $s \geq \tau$
Status	Text	PASS, FAIL or ERROR	Operator action: let through, re-inspect, or check the camera or file
Defects	Class; confidence 0 to 1; box corners in px; area as a fraction of the frame	Five classes	What is wrong and where to look
Timing	Milliseconds	Positive	Load, preprocess, inference, postprocess and total time; checks the line budget
Warnings	Text	Wrong size, very dark (mean below 15/255), very bright (above 240/255), near-uniform	The input is unlike the training data

Each frame produces a JSON report and an annotated PNG. A batch also gets a summary and an HTML index. Invalid inputs never stop a batch. A file that is not an image, or cannot be decoded, gets status ERROR with the reason. Grey or RGBA images are converted to RGB. The command-line tool exits with code 2 if any frame was an ERROR, and 1 for invalid arguments. A test (`test_cli_inspect_mixed_inputs`) checks this with a mix of good, corrupt and non-image files. The inputs, outputs and exceptions of every public function are documented in its docstring, and collected in the HTML documentation (Appendix A).

2.2 The data

The data is the Roboflow "Pork Rasher Error(Packaging)" export, version 4, under CC BY 4.0 [3]. It holds 3,549 images from 1,272 distinct frames of an overhead conveyor video, with bounding boxes for the five defect classes. Roboflow had already resized every frame to 720 x 540 using "fill with centre crop", so tray edges can be cut off. Pixel values were not normalised: that is done inside the pipeline. Each training frame has three augmented copies with exposure randomly shifted by up to ±10%. The unit's dataset slide also mentions saturation changes, but the export's own README lists exposure only.

I derived the frame label from the boxes: FAIL if any defect box is present, PASS if none. This makes PASS the minority, with 684 of 3,349 training images, 14 of 120 validation and 12 of 80 test. Two classes dominate. Unsealed film appears in 1,055 training images and packaging errors in 1,541. Twisted meat, loose meat and wrinkle each appear in fewer than 70, and wrinkle in no test frame at all. The classes also differ greatly in size. The median packaging-error box covers 0.38% of the frame (about 37 x 37 px), while the median unsealed box covers 10.9%.

2.3 Leakage in the provided split, and the blocked split

The provided test frames are interleaved with training frames from the same video. Of the 80 test frames, 23 have a training frame within one frame number, and all 80 have one within 30 (Figure 1). Neighbouring frames show the same physical tray at a slightly different position. A model tested on them is partly being tested on trays it trained on, which is a form of leakage [4].

To set a safe gap, I measured how frame similarity changes with frame distance. The correlation between small thumbnails rises and falls with a period of 11.9 frames, found from the FFT of the curve (Appendix B). That period is the spacing between trays on the belt. Frames 30 or more apart, about 2.5 tray spacings, therefore show different trays.

The blocked split groups frames into contiguous blocks of 200 frame numbers and assigns whole blocks to train, validation or test, as recommended for temporally structured data [5]. Any frame within 30 frame numbers of a higher-priority split is dropped. Among 500 random block assignments, I kept the one whose validation and test PASS rate and class mix were closest to the whole dataset. The choice used labels only, never model results. The result is 869 training frames (2,433 images with copies), 192 validation and 174 test frames, of which 37 and 34 are PASS. No test frame has a training frame within 30 frame numbers. Validation and test use one copy of each frame, so no frame is counted twice.

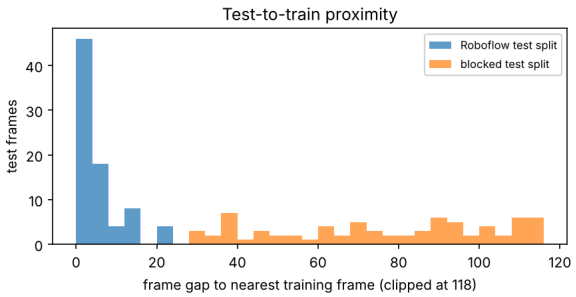


Figure 1. Frame gap from each test frame to its nearest training frame. Every provided test frame sits within 30 frames of a training frame, and many within one. The blocked split removes this.

3 Computational approach

Figure 2 shows the whole system. Every model, simple or deep, is reduced to the same form: one defect score s in [0, 1] per frame, and the frame FAILs when s ≥ τ. This lets every model be judged by the same rule and the same threshold procedure.

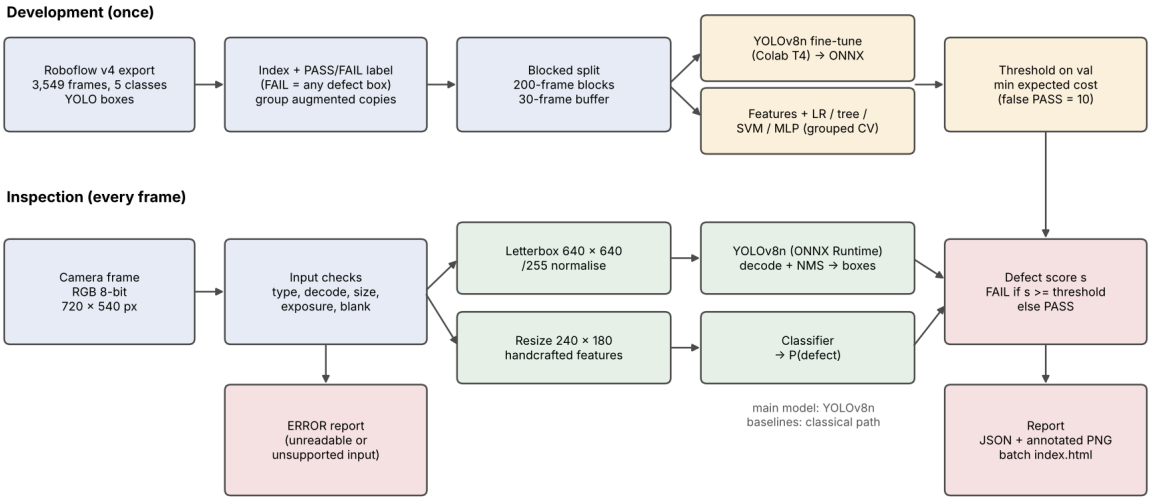


Figure 2. PorkSort architecture. Development (top) runs once; inspection (bottom) runs on every frame. YOLOv8n is the selected model; the classical path is the baseline family it was compared against.

3.1 Classical models on handcrafted features

The classical models cannot use raw pixels: a frame has over a million values, and there are only 869 distinct training frames. Each frame is therefore resized to 240 x 180 px and summarised by 1,439 features in six groups:

- a joint HSV colour histogram (128 bins), since meat, film and belt differ in colour;
- the share, centroid and spread of meat-coloured pixels, to locate the tray;
- the share of bright, unsaturated pixels on a 4 x 3 grid, because loose or wrinkled film reflects light differently;
- mean edge strength on a 6 x 4 grid;
- a local binary pattern texture histogram [6];
- and a histogram of oriented gradients (HOG) [7], which captures shape.

Four classifiers were trained on these features, each the classification counterpart of a regression model from the unit [17]: logistic regression (linear models, 5.1), a decision tree (5.2), a support vector machine with an RBF kernel (5.3) and a multilayer perceptron (5.4). All were implemented in scikit-learn [8], with scikit-image [9] for the features. Features were standardised to zero mean and unit variance before the logistic regression, SVM and MLP. Without this, large-range features dominate the SVM's distance calculation and make gradient descent badly conditioned. Classes were weighted by inverse frequency, so the 20% PASS minority was not ignored.

Hyperparameters were chosen by five-fold cross-validation inside the training set, scored by ROC AUC. The folds were grouped by 200-frame block (StratifiedGroupKFold), so the augmented copies of one frame never sat on both sides of a fold (unit topic 5.6). The chosen settings are in Appendix C.

3.2 YOLOv8n detector

The brief asks for defects to be detected and localised. The smallest class is about 37 px wide, and Section 4 shows that global features cannot see it. A detector that looks at every part of the image is the natural tool. I chose YOLOv8n, the smallest model in the Ultralytics YOLOv8 family [10], [11], for three reasons. It is fast enough for a CPU on the line. Its pretrained weights transfer to a small dataset. And it outputs boxes, which explain each FAIL to the operator.

The network is a convolutional neural network. Its backbone turns the image into feature maps at three scales, and a detection head predicts a box and a score for each of the five classes at each of 8,400 grid positions (for a 640 px input). Predictions below a confidence threshold are discarded. Non-maximum suppression (NMS) then removes overlapping duplicates: it keeps the highest-scoring box and deletes any box of the same class whose intersection over union (IoU) with it exceeds 0.6.

I fine-tuned weights pretrained on the COCO dataset. Training used a 640 px input, with the 720 x 540 frames letterboxed to 640 x 480 plus grey padding. It ran for up to 60 epochs with early stopping after 20 epochs without improvement, batch size 16, seed 0, and Ultralytics' default augmentation. Training ran on a free Google Colab T4 GPU for 22 minutes. Early stopping ended it at epoch 39 and kept the weights from epoch 19, the best on validation mAP. The trained network was exported to ONNX. Inspection runs it with ONNX Runtime on a CPU [12]. The letterboxing, output decoding and NMS are my own NumPy code, so the tool needs neither PyTorch nor a GPU. The deployed system runs the network at a 480 px input, which Section 5.4 shows keeps its accuracy at 59% of the CPU time.

3.3 Choosing the threshold by cost

A threshold of 0.5 has no special meaning here. The two errors have different costs: a defective pack that passes risks a recall, while a good pack that fails only costs a re-check. I chose τ to minimise the expected cost per frame on the validation set:

$$C(\tau) = [\ r \cdot FN(\tau) + FP(\tau)\ ] / N$$

Here FN counts defective frames passed, FP counts good frames failed, and N is the number of frames. I set the cost ratio r = 10, treating a shipped defect as ten times worse than a re-check. Section 5 tests how much the decision depends on this choice. The candidate thresholds are the midpoints between consecutive distinct scores, so the search finds the exact minimum on the data. Ties go to the lower threshold, because a false PASS is the more expensive mistake.

3.4 Assumptions

- The annotations are correct and complete for what is visible in each frame.
- New frames come from the same line, camera and lighting.
- All five defect classes are equally serious.
- Each frame is judged on its own.

4 Validation and verification

4.1 What was validated and verified

Validation asks whether the decisions are right. I compared the PASS/FAIL decision for each of the 174 withheld test frames of the blocked split against its label, once, after every choice had been fixed.

Verification asks whether the code does what I intended. The repository has 49 automated tests (Appendix D). Four compare the code with an answer known in advance: the YOLO decoder recovers a hand-encoded box to within 1e-5, the FLOP formula matches a direct loop count, the threshold search matches a 20,001-point brute-force search, and the optimised histogram is bit-identical to NumPy's `histogramdd`.

I also compared my ONNX pipeline with an established implementation, Ultralytics' own predictions, on 60 test frames. Of the 41 boxes Ultralytics found with confidence of at least 0.25, mine matched 34 at IoU ≥ 0.9 (median IoU 0.94). Frame scores differed by 0.04 on average and 0.30 at most. Two checks place the cause in the input, not the decoding. Switching my resizing to OpenCV's, as Ultralytics uses, changed the mean difference only from 0.0392 to 0.0389. Moving the padding from centred to top-aligned, with nothing else changed, shifted scores by 0.024 on average. Ultralytics' PyTorch predictor feeds a 640 x 480 rectangle, while the exported ONNX model needs a padded 640 x 640 square, and the network is sensitive to that. All results below come from the ONNX pipeline that is actually deployed.

4.2 Why this method is appropriate

Withheld data is the only direct test of performance on unseen frames, and the blocked split keeps every test tray out of training (Section 2.3). Rates estimated from 34 PASS frames are uncertain, so each is reported with a Wilson score 95% interval [13], which unlike the normal approximation stays valid at 0% and 100%. ROC AUC intervals come from 2,000 bootstrap resamples of the test frames.

4.3 Metrics and acceptance criteria

FAIL is the positive class. The primary metric is the **escape rate**, FN / (TP + FN), the share of defective frames passed. The secondary metric is the **false reject rate**, FP / (FP + TN), the share of good frames sent for re-checking. ROC AUC measures how well a model ranks frames across all thresholds. Accuracy is not used: always answering FAIL scores 80.5% on the blocked test. Before seeing any detector results, I set two acceptance criteria: an escape rate whose upper 95% bound is at most 5%, and a false reject rate of at most 30%.

4.4 Did validation influence model selection?

Yes. Cross-validation inside the training set chose each classical model's hyperparameters. The validation set chose the threshold, the input size and the final model. The test set was used once, at the end.

4.5 Results

Table 1. Blocked test set (174 frames, 34 PASS), thresholds chosen on validation at r = 10. Intervals are 95%.

Model	ROC AUC (95% CI)	Threshold	Escape rate	False reject rate	Cost per frame
Always FAIL (baseline)	0.500	0	0.000	1.000	0.195
Colour-histogram logistic regression	0.739 (0.652 to 0.822)	0.205	0.000	0.941	0.184
Logistic regression	0.739 (0.649 to 0.823)	0	0.000	1.000	0.195
Decision tree	0.717 (0.621 to 0.807)	0.147	0.029	0.971	0.420
SVM (RBF)	0.724 (0.633 to 0.806)	0	0.000	1.000	0.195
MLP	0.717 (0.618 to 0.805)	0	0.000	1.000	0.195
YOLOv8n, 640 px input	0.859 (0.773 to 0.926)	0.004	0.000	1.000	0.195
YOLOv8n, 480 px input (selected)	**0.887 (0.810 to 0.947)**	0.007	0.000	1.000	0.195

YOLOv8n ranks frames far better than any classical model (Figure 3). On the same test frames its AUC beats logistic regression's by 0.148, with a paired bootstrap 95% interval of 0.068 to 0.235, so the gain is not sampling noise. Comparing PASS frames with frames containing each defect, the detector's AUC is 0.95 for unsealed film, 0.84 for packaging errors, 0.71 for loose meat (6 frames) and 0.97 for twisted meat (4 frames). For logistic regression the same figures are 0.86, 0.67, 0.51 and 0.49. Global features see only the large, tray-sized defect, while the detector also finds most small ones.

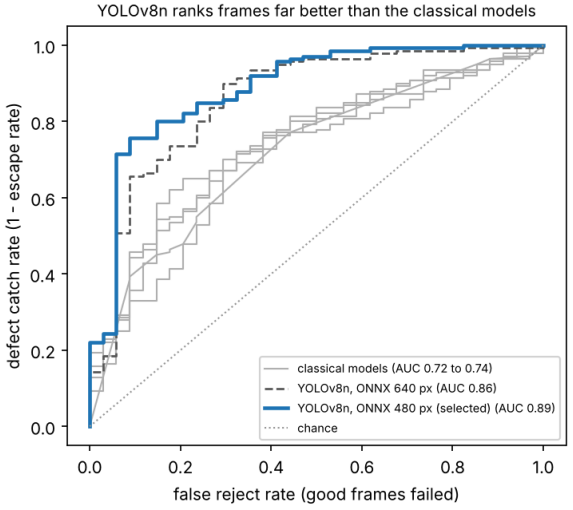


Figure 3. ROC curves on the blocked test set. The selected detector (blue) dominates every classical model (grey).

Ranking well is not enough at r = 10. With 80% of frames defective, passing a good frame saves one re-check, while each escape costs ten. On validation the lowest-scoring frames still include defective ones, among them five wrinkle frames: the training set holds only three wrinkle frames, so the model never learned that class. The cost-optimal threshold therefore fails every frame, for every model, and the system does not meet the acceptance criteria. Table 2 shows where the detector does pay off.

Table 2. Selected detector: operating points chosen on validation, evaluated on test.

Cost ratio r	Threshold	Escape rate (95% CI)	False reject rate (95% CI)	Cost per frame (re-check all: 0.195)
1	0.128	3.6% (1.5% to 8.1%)	44% (29% to 61%)	0.115
2 to 50	0.007	0%	100%	0.195

At r = 1 the detector passes 19 of the 34 good frames automatically and lets 5 of the 140 defective frames through, cutting the cost per frame by 41%. Logistic regression fails every frame at every r. Figure 4 shows typical decisions at r = 1. Most escapes are small packaging errors at tray corners and loose pieces at the frame edge. Several false rejects are trays whose film looks lifted but whose frame carries no label, which may be label ambiguity rather than model error.

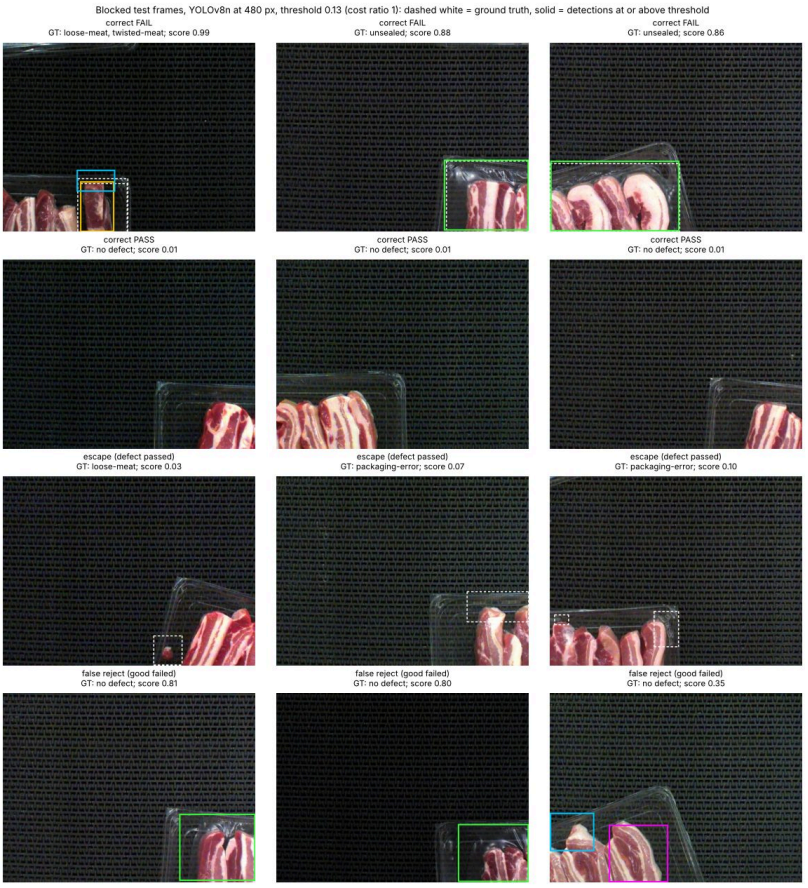


Figure 4. Blocked test frames at the r = 1 threshold: correct decisions, escapes and false rejects. Dashed white boxes are labels; solid boxes are detections.

The split matters as much as the model. On the leaky Roboflow test split, logistic regression scores an AUC of 0.808 and the SVM 0.826, against 0.739 and 0.724 on the blocked split. A random split would have overstated their performance by 0.07 to 0.10. I did not train a second detector on the Roboflow split to measure the same effect for YOLO.

4.6 What the validation cannot establish

- Performance on another line, camera, lighting setup or product.
- Behaviour per tray: a defect visible in one frame may be out of view in the next.
- Recall for rare classes: wrinkle has no test frames, and loose and twisted meat have four to six.
- The effect of label errors, such as a defect just out of frame labelled PASS.
- Whether r = 10 reflects the plant's real costs.

5 Alternatives, optimisation and performance

5.1 Comparison of alternatives

Table 3. Alternatives on the criteria the brief lists. Times are per frame on the M4 laptop CPU unless marked.

Model	Blocked test AUC	Time per frame	Model size	Interpretability	Localises defects
Logistic regression	0.739	2.8 ms features (optimised) + 0.25 ms (container)	about 35 kB	High: one weight per feature	No
Decision tree (depth 5)	0.717	features + 0.11 ms (container)	under 10 kB	High: readable rules	No
SVM (RBF)	0.724	features + 2.0 ms (container)	1,606 support vectors	Low	No
MLP (64 hidden units)	0.717	features + 0.31 ms (container)	about 0.75 MB	Low	No
YOLOv8n, 480 px	**0.887**	36.7 ms inference, 41 ms end to end	12.7 MB (fp32), 6.4 MB (fp16)	Medium: boxes show why	Yes

I selected YOLOv8n at 480 px. It has the highest AUC, with a paired interval that excludes zero, it is the only model that shows the operator where the defect is, and it is fast enough (Section 5.5). Its costs are a larger model, GPU training and less transparent internals. It is selected as the best model available, not as one that meets the acceptance criteria.

5.2 Profiling and optimising the classical pipeline

Feature extraction, not the classifier, dominates the classical pipeline's time. Profiling `features.extract` with line_profiler on the M4 showed RGB-to-HSV conversion at 36% of the time, the 3D colour histogram at 14%, texture features at 13%, HOG at 11% and resizing at 10%. I made four changes, each checked for its numerical effect:

1. I replaced scikit-image's float64 HSV conversion with PIL's 8-bit conversion, written in C. Values now differ from the reference by at most one 8-bit step (1/255), far finer than the histogram bins of 1/8 and 1/4. A test checks this bound.
2. I computed each pixel's histogram bin directly and counted with `bincount`. A test confirms the counts are bit-identical to `histogramdd`.
3. Dropping one feature group at a time showed only colour and HOG matter: without them validation AUC fell by 0.026 and 0.036, while each other group changed it by under 0.003. I removed the other four.
4. Resizing is 2D interpolation (unit topic 3.5). A study over three working sizes and four interpolation methods (Appendix B) found box (area-averaging) filtering best at every size. Nearest-neighbour keeps one pixel in every few, so the belt's fine weave aliases into noise, while box filtering averages it out. I chose box filtering at 120 x 90 px.

On the M4, extraction time fell from 13.9 ms to 9.0, 4.9 and finally 2.8 ms per frame, 4.9 times faster in total (7.3 times in the development container). Validation AUC rose from 0.718 to 0.758 and test AUC from 0.739 to 0.748, so the faster pipeline is no less accurate.

5.3 YOLOv8n: FLOPs, arithmetic intensity and the roofline

I counted the FLOPs of every convolution in the ONNX graph as 2 x C_out x H_out x W_out x (C_in / groups) x k^2. This gives 8.64 GFLOPs per 640 px frame, within 5% of the 8.2 GFLOPs Ultralytics reports. I did not trace the remaining difference, which comes from how the two tools count layers. Assuming each layer reads its weights and input and writes its output once, the network moves 153 MB per frame, an arithmetic intensity of 57 FLOP/byte. Individual layers range from 0.5 to 174 FLOP/byte.

For the roofline model [14], I estimated the M4's peak with the unit's formula: cores x clock x vector lanes x FMA factor x floating-point pipes. For the 4 performance cores this gives 4 x 4.4 GHz x 4 x 2 x 4 ≈ 563 GFLOP/s [15]. The 4 pipes per core is an estimate from microarchitecture analyses, not an Apple figure. With about 120 GB/s of memory bandwidth, the machine balance is 4.7 FLOP/byte. The Colab T4's balance is about 25 FLOP/byte (8.1 TFLOP/s, 320 GB/s) [16]. At 57 FLOP/byte the network is compute-bound on both (Figure 5). On the M4 it achieved 140 GFLOP/s, about a quarter of the estimated roof. Likely reasons are that the virtual machine's four cores may not all be performance cores, small layers use the vector units poorly, and activations, concatenation and decoding add work my count leaves out.

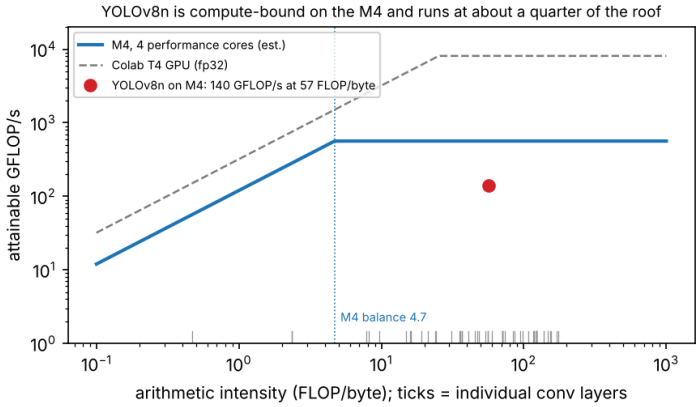


Figure 5. Roofline for the M4 and the T4. Ticks mark individual convolution layers; the red point is the measured whole-network performance on the M4.

5.4 Precision, threads and input size

Half precision. Exporting the network in fp16 halves the file from 12.8 to 6.4 MB and cut M4 inference at 640 px from 61.9 to 42.9 ms, because ARM vector registers hold eight fp16 values instead of four fp32. Over 60 test frames the largest change in a frame score was 0.0008. That is consistent with fp16 rounding: fp16 keeps 11 significant bits, so its unit roundoff is 2^-11 ≈ 0.0005. No PASS/FAIL decision changed, and test AUC was identical.

Threads. Inference at 640 px took 207 ms on one thread and 62 ms on four, a 3.3 times speed-up on four cores.

Input size. The network was trained at 640 px but can run at other sizes (Figure 6). Validation AUC was 0.848, 0.875, 0.870 and 0.851 at 320, 480, 640 and 800 px, while M4 time grew roughly with pixel count: 17.7, 36.7, 61.9 and 98.5 ms. I chose 480 px on validation: the same AUC as 640 px within noise, at 59% of the time. Its higher test AUC (0.887 against 0.859) is not claimed as a real gain. Running at 800 px did not help the small packaging errors, which suggests training at a higher resolution, not just running at one.

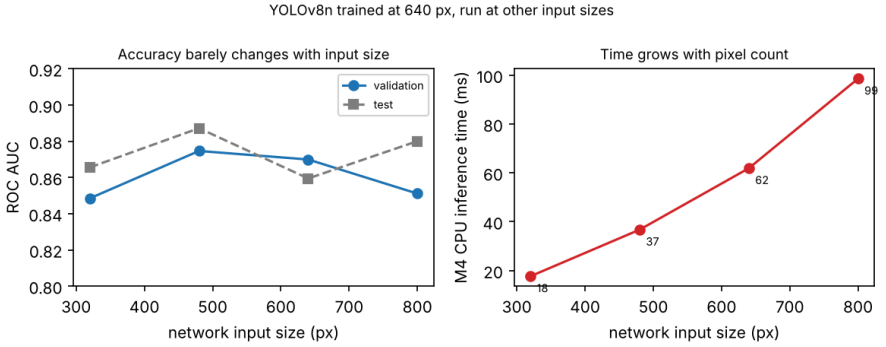


Figure 6. YOLOv8n trained at 640 px and run at other input sizes: accuracy barely changes, while time grows with pixel count.

5.5 Is it fast enough for the line?

A line running 120 to 200 packs per minute leaves 300 to 500 ms per pack if one frame per pack is judged. At 41 ms end to end on the M4, the selected system is 7 to 12 times inside that budget. Judging every frame, at 11.9 frames per tray, leaves only 25 to 42 ms per frame, which the 480 px model only just meets. The 320 px model (21 ms) fits comfortably, at a cost of 0.027 validation AUC.

5.6 Sensitivity to the cost ratio, and repeatability

Sweeping r from 1 to 50 (Appendix B) shows the detector's operating point changing only between r = 1 and r = 2, while every classical model fails all frames throughout. Their weakness is in ranking, not in the choice of r. The MLP's test AUC over five random seeds was 0.716 ± 0.008, converging in 50 to 70 iterations under early stopping. YOLOv8n was trained once, so its run-to-run spread is unknown.

6 Limitations and lessons learned

6.1 Limitations

- **It does not meet the acceptance criteria.** At r = 10 no model beats re-checking every pack. The detector is useful as a prioritisation aid with boxes, not yet as an automatic gate.

- **Per-frame decisions.** A tray is in view for about 12 frames and is often only partly visible, so a frame can be labelled PASS while its defect is just out of view. A deployed system should track each tray and fail it if any of its frames fails.
- **Rare classes.** Wrinkle has three training frames and no test frames, and loose and twisted meat have four to six test frames each, so nothing is claimed about their recall.
- **One recording.** All frames come from one line, camera and lighting setup, with an unknown frame rate. Thirty-two frame names carry two different images, which suggests two recording sessions with overlapping numbering.
- **One training run.** YOLOv8n was trained with a single seed, so its run-to-run variation is unknown.

6.2 Lessons learned

- **How the data is split decides what is measured.** The provided split overstated classical AUC by up to 0.10, because test trays had been seen in training.
- **Ranking is not deciding.** An AUC of 0.89 still fails a 10 to 1 cost test when 80% of frames are defective. The decision has to be judged at its operating point and cost, not by AUC alone.
- **Match the method to the scale of the defect.** Global features found unsealed trays (AUC 0.86) but not small local defects (0.49 to 0.67). The detector reached 0.71 to 0.97 on the same classes.
- **Profile before optimising.** The slowest step was colour conversion, not the model. Fixing it and removing features that did not earn their time made extraction 4.9 times faster on the M4 with no loss of accuracy.

6.3 Proposed improvements

1. Train at a higher input resolution, such as 960 px, to target the small packaging errors that cause most escapes.
2. Collect more wrinkle, loose-meat and twisted-meat examples, or merge them into one placement class that can be validated.
3. Track trays across frames and combine their decisions.
4. Train with several seeds to measure run-to-run variation, and replace r = 10 with costs measured at the plant.

7 AI use and reflection

Tools and tasks. I used Claude (Anthropic) through the Claude app, linked to my laptop and a cloud workspace. It wrote most of the code in the repository: the package, the analysis scripts, the tests and the Colab training notebook. It ran the data audit and the analyses, and produced the first draft of this report from a section plan built against the rubric. Google Colab provided the GPU for training.

Level of contribution. High. I estimate the AI produced about [confirm]% of the code and the first draft of the report text. I set the problem, made or approved each engineering decision, ran the training, and revised the report.

Why I used it. The project needed a toolchain I had not used before (YOLO, ONNX, profiling of image code), and it fell in a week with two other tests.

How I checked its output. 49 automated tests, four against answers known in advance. An independent comparison of the ONNX pipeline against Ultralytics' own predictions, including two experiments to explain the differences. Looking at sample frames and every figure, not only the numbers. Tracing each number in this report to a file in `results/`.

Decisions that remained mine. The dataset and scope, the per-frame PASS/FAIL goal, and training on Colab rather than locally. The AI proposed the blocked split, the cost-based threshold, r = 10 and the acceptance criteria. I reviewed and accepted each one, and I am responsible for them.

Errors and unhelpful output.

- Its first frame-similarity analysis compared each gap against random pairs ±2 standard deviations, a band too wide to show anything. The plot revealed a repeating pattern instead, and an FFT of the curve gave the 11.9-frame tray spacing.
- Its Colab notebook failed on first run: the code folder had the same name as the Python package and hid it. The fix was to import the package from its source folder.
- It delivered an out-of-date copy of the notebook and code to my folder, caught by checking file sizes.
- Two of its tests failed on first run because of bugs in the tests themselves, and a draft sentence assumed a 30 fps camera that the data does not support.

Effect on my learning. The AI made a far more thorough project possible in the time available. The risk is understanding the result less well than if I had built it myself. To reduce that, [confirm: e.g. I worked through the split design, the cost threshold, NMS and the roofline with guided explanations until I could explain them without notes].

References

1. "Proseal launch for GT2X Twin E-seal," *BakeryandSnacks.com*, Jul. 2012. [Online]. Available: bakeryandsnacks.com
2. "Harpak ULMA launches high speed grind line tray sealing application powered by Mondini Sinfonia," *Business Wire*, Jan. 2026. [Online]. Available: businesswire.com
3. "Pork Rasher Error(Packaging)," version 4, Roboflow Universe, workspace hello-2aqe0, licensed CC BY 4.0, 2025. [Online]. Available: universe.roboflow.com
4. S. Kaufman, S. Rosset, C. Perlich and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, 2012.
5. D. R. Roberts *et al.*, "Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure," *Ecography*, vol. 40, no. 8, pp. 913-929, 2017.
6. T. Ojala, M. Pietikäinen and T. Mäenpää, "Multiresolution gray-scale and rotation invariant texture classification with local binary patterns," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 24, no. 7, pp. 971-987, 2002.
7. N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. IEEE CVPR*, 2005, pp. 886-893.
8. F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825-2830, 2011.
9. S. van der Walt *et al.*, "scikit-image: Image processing in Python," *PeerJ*, vol. 2, e453, 2014.
10. J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE CVPR*, 2016, pp. 779-788.
11. G. Jocher, A. Chaurasia and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: github.com/ultralytics/ultralytics
12. ONNX Runtime developers, "ONNX Runtime," Microsoft. [Online]. Available: onnxruntime.ai
13. E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209-212, 1927.
14. S. Williams, A. Waterman and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65-76, 2009.
15. "Apple Silicon: M4 chip specs," *Low End Mac*, 2025. [Online]. Available: lowendmac.com
16. NVIDIA, "NVIDIA T4 Tensor Core GPU." [Online]. Available: nvidia.com
17. MMA3001 teaching team, *MMA3001 Numerical Methods and Machine Learning unit notes*, Weeks 2.1, 2.2, 3.5 and 5.1 to 5.6, Monash University, 2026.

Appendix A: Repository guide

Repository: github.com/hashim-edu/mma3001-project-pork_sort. The README explains the purpose, inputs and outputs, installation, and how to reproduce every result. To reproduce:

1. Clone the repository and install it with `pip install -r requirements.txt` then `pip install -e .`
2. Unzip the Roboflow export to `data/pork_v4/`.
3. Run the scripts in order: `01_data_audit.py`, `02_classical_models.py`, the Colab notebook `03_train_yolo_colab.ipynb`, `04_evaluate_and_compare.py`, `05_performance.py` and `06_figures.py`.

Folder	Contents
`src/porksort/`	Package: dataset, splits, features, models, decision, evaluate, detector, perf, report, cli
`scripts/`	Numbered analysis scripts that write to `results/`
`notebooks/`	Colab training notebook
`splits/`	The committed blocked split, so every machine uses the same frames
`results/`	Metrics (JSON, CSV), profiles and figures used in this report
`tests/`	pytest suite on synthetic data; needs no dataset
`docs/`	HTML API documentation (pdoc, Google-style docstrings) and the pytest HTML test report

The inspection tool runs with `python -m porksort inspect <frames> --out reports/`. The `--cost-ratio` option picks the threshold chosen on validation for that cost ratio, for example `--cost-ratio 1`. Code is under the MIT licence; the data remains CC BY 4.0 and is not redistributed.

Appendix B: Additional figures

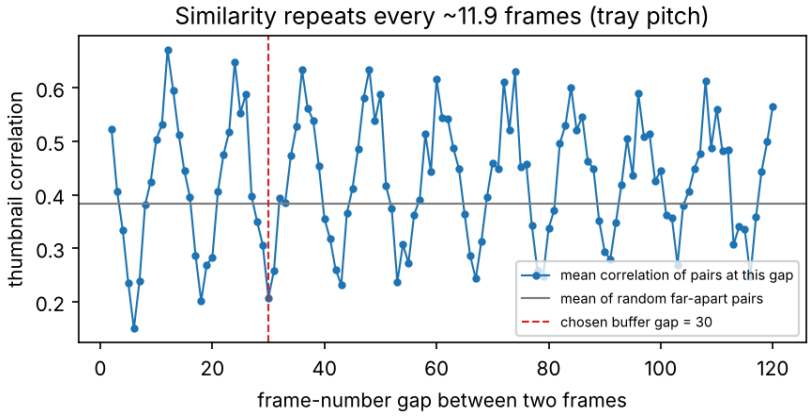


Figure B1. Correlation of frame thumbnails against frame gap. The curve repeats every 11.9 frames, the spacing between trays; the 30-frame buffer spans about 2.5 trays.

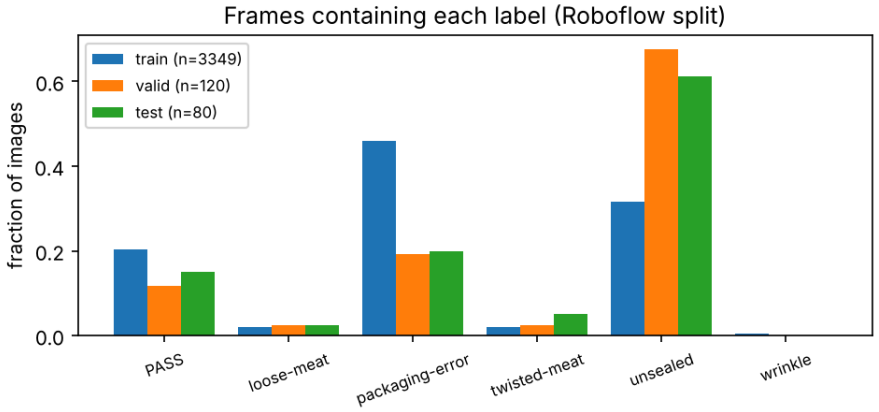


Figure B2. Share of frames containing each label in the provided splits. PASS is a minority and three classes are rare.

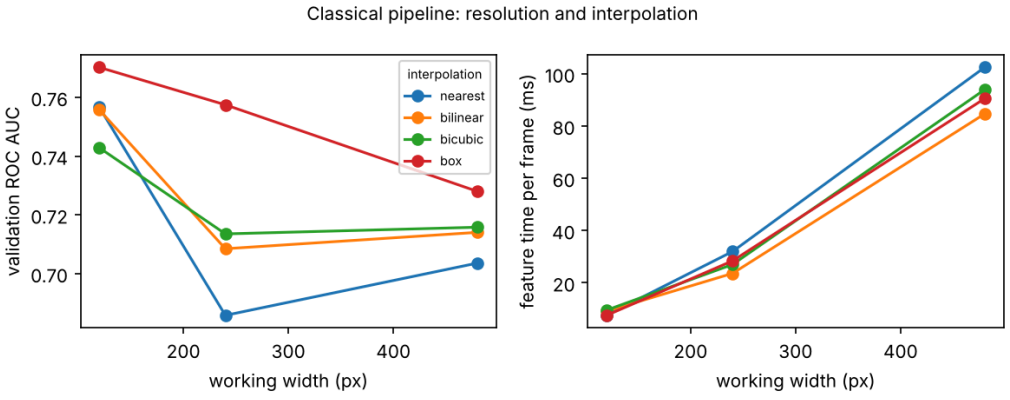


Figure B3. Classical pipeline: validation AUC (left) and feature time (right) against working width and interpolation method. Box filtering is best at every size; time grows with pixel count.

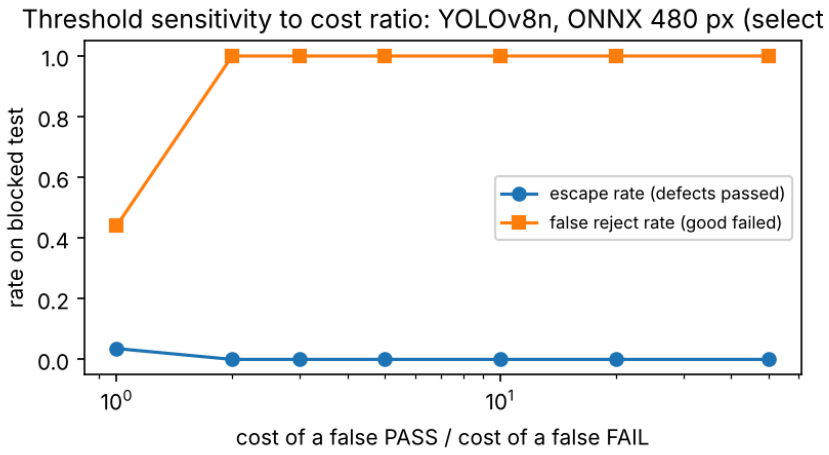


Figure B4. Selected detector: test escape rate and false reject rate at the threshold chosen on validation for each cost ratio. Only at r = 1 does the threshold pass any frames.

Appendix C: Chosen settings

Model	Settings chosen by grouped cross-validation
Colour-histogram logistic regression	C = 0.01, balanced class weights
Logistic regression	C = 0.001, balanced class weights
Decision tree	maximum depth 5, at least 20 samples per leaf (18 leaves)
SVM	RBF kernel, C = 3, gamma = 1e-4 (1,606 support vectors)
MLP	one hidden layer of 64 units, L2 penalty alpha = 1.0, early stopping
YOLOv8n	640 px input, up to 60 epochs, patience 20, batch 16, seed 0, COCO-pretrained; stopped at epoch 39 with the weights from epoch 19; deployed at a 480 px input
Decision threshold	chosen on validation at cost ratio r = 10

Appendix D: Verification tests

Test file	Tests	What they establish
`test_dataset.py`	10	Label files are parsed and malformed ones rejected with the file and line; augmented copies group to one source frame
`test_splits.py`	6	Whole blocks stay together, no test frame lies within the gap of a training frame, evaluation uses one copy per frame, the split is reproducible
`test_decision_evaluate.py`	8	The PASS/FAIL rule; the threshold search equals a brute-force search and never rises as the cost ratio rises; Wilson intervals match known values
`test_features_detector.py`	13	Feature vectors have a fixed length; letterbox, IoU and NMS give known answers; the decoder recovers a hand-encoded box; the FLOP formula matches a loop count; the optimised histogram and HSV conversion match the references
`test_onnx_pipeline.py`	4	A small synthetic ONNX network runs end to end in fp32 and fp16, and the graph FLOP profiler matches the formula
`test_report_cli.py`	5	Reports are written; corrupt, non-image and invalid inputs become ERROR reports without stopping a batch; thresholds are looked up by model and cost ratio
Doctests in `src/`	3	Worked examples in the docstrings

Appendix E: Environment

Development and classical models: Python 3.13, scikit-learn 1.9.1, scikit-image 0.26.0, NumPy 2.5.3, pandas 3.0.5, ONNX Runtime 1.29.0, Pillow 12.3.0. Timing on the target laptop: Apple MacBook with M4 chip, in a 4-core Linux virtual machine with Python 3.10. Training: Google Colab with an NVIDIA T4 GPU, Ultralytics 8.4.185, PyTorch 2.11.0 (CUDA 13.0), Python 3.13.